

Universidad Latina de Costa Rica

Programa Tienda Tecnológica

Proyecto Final

Estudiantes:

Steven Kaled Castro Castro

José Pablo Matarrita Quirós

Curso:

Programación IV

Docente:

Granados Venegas Osvaldo Fabian

Contenido

1. Introducción.....	3
2. Objetivos.....	3
3. Descripción del sistema	4
4. Requerimientos.....	5
5. Tecnologías utilizadas.....	6
6. Arquitectura y estructura del proyecto	7
7. Base de datos	9
8. Instalación y ejecución	10
9. Funcionalidades principales	12
10. Pruebas.....	14
11. Mantenimiento y solución de problemas.....	14
12. Conclusiones.....	15
13. Referencias	16

1. Introducción

Tienda Tecnológica es un sistema de comercio electrónico (e-commerce) orientado a la venta de productos tecnológicos (celulares, computadoras, accesorios, etc.). Permite a los clientes explorar un catálogo de productos, agregarlos a un carrito de compras, seleccionar un método de pago y finalizar su pedido, mientras que desde el lado administrativo se gestionan productos, inventario, categorías, proveedores y pedidos.

Antes del desarrollo del sistema, la gestión de ventas y de catálogo se realizaba de forma manual o dispersa en distintas herramientas, lo que dificultaba llevar un control centralizado del inventario, los precios, los descuentos y el historial de compras de los clientes.

El propósito del sistema es centralizar en una sola plataforma web todo el proceso de venta: desde la publicación del producto hasta la confirmación del pedido y el pago, brindando una experiencia de compra clara tanto para el cliente como para el administrador de la tienda.

2. Objetivos

2.1. Objetivo general

Desarrollar un sistema web de comercio electrónico para la Tienda Tecnológica que permita gestionar el catálogo de productos, el proceso de compra y la administración de pedidos de manera centralizada y segura.

2.2. Objetivos específicos

- Implementar un módulo de catálogo de productos con categorías, subcategorías, marcas e imágenes.

- Desarrollar un carrito de compras y un proceso de checkout con selección de método de pago y cálculo automático de impuestos (IVA).
- Diseñar una base de datos relacional que soporte clientes, productos, pedidos, pagos, inventario.
- Implementar autenticación y control de acceso de usuarios mediante JWT (JSON Web Token).

3. Descripción del sistema

El sistema permite a un cliente registrarse, iniciar sesión, navegar por categorías y subcategorías de productos, buscar productos, agregarlos a favoritos o al carrito, y finalizar la compra generando un pedido con su respectivo pago. Del lado administrativo, permite gestionar productos, inventario, bodegas, proveedores, descuentos, garantías y el estado de los pedidos.

3.1. Principales funcionalidades

- Registro e inicio de sesión de clientes.
- Catálogo de productos con búsqueda, categorías y subcategorías.
- Carrito de compras (agregar, quitar, modificar cantidades).
- Lista de favoritos / deseos.
- Checkout con selección de método de pago y cálculo de IVA.
- Historial y confirmación de pedidos.
- Gestión administrativa de productos, inventario, proveedores y descuentos.

3.2.Usuarios a los que está dirigido

- Clientes: personas que navegan el catálogo y realizan compras.
- Administradores / empleados: gestionan productos, inventario y pedidos.

4. Requerimientos

4.1.Hardware (mínimos recomendados)

- Procesador de doble núcleo, 2.0 GHz o superior.
- 4 GB de memoria RAM (8 GB recomendado para desarrollo).
- 2 GB de espacio libre en disco.
- Conexión a internet para consumir el API y descargar dependencias.

4.2.Software

- Sistema operativo: Windows 10/11 (o Linux/Mac para el frontend Angular).
- Visual Studio 2022 (o superior) para el backend en .NET.
- Visual Studio Code para el frontend en Angular.
- Node.js y npm para compilar y ejecutar el proyecto Angular.
- SQL Server (Express o superior) y SQL Server Management Studio (SSMS).
- Git, para el control de versiones.

4.3. Requerimientos funcionales principales

- El sistema debe permitir el registro y autenticación de usuarios.
- El sistema debe mostrar el catálogo de productos con sus imágenes, precios y existencias.
- El sistema debe permitir agregar productos al carrito y calcular el total con impuestos.
- El sistema debe registrar el pedido, el detalle de productos y el pago asociado.

5. Tecnologías utilizadas

Tecnología	Uso
C# / .NET 10	Desarrollo del backend (API REST)
Angular	Desarrollo del frontend (interfaz web)
TypeScript / SCSS	Lógica y estilos del frontend
SQL Server	Motor de base de datos
Entity Framework Core	Acceso a datos (ORM)
JWT (JSON Web Token)	Autenticación y control de acceso
AutoMapper	Mapeo entre entidades y DTOs
Swagger / Swashbuckle	Documentación y pruebas del API
Visual Studio / VS Code	Entornos de desarrollo
Git / GitHub	Control de versiones

6. Arquitectura y estructura del proyecto

El sistema está dividido en dos grandes partes: un backend construido en .NET con arquitectura en capas, y un frontend en Angular que consume el API mediante peticiones HTTP.

6.1. Capas del backend

- Tienda.API: expone los controladores y endpoints REST que consume el frontend.
- Tienda.LogicaNegocio: contiene las reglas y procesos de negocio del sistema.
- Tienda.AccesoDatos: contiene el contexto de Entity Framework y el acceso a la base de datos.
- Tienda.Dominio: contiene las entidades y modelos del dominio del negocio.
- Tienda.Utilidades: contiene clases de apoyo comunes a los demás proyectos (por ejemplo, la clase Respuesta estándar del API).

6.2. Estructura del frontend (Angular)

- Componentes/: componentes visuales de la tienda (carrito, catálogo, detalle de producto, encabezado, etc.).
- app/services/: servicios que consumen el API (productos, carrito, pedidos, autenticación, etc.).
- app/model/: interfaces que representan los datos que viajan entre el frontend y el API.

6.3. Comunicación entre componentes

El navegador ejecuta la aplicación Angular, que realiza peticiones HTTP (GET, POST, PUT, DELETE) hacia el API REST desarrollado en .NET. El API valida la petición (incluyendo el token JWT del usuario cuando aplica), procesa la lógica de negocio correspondiente y consulta o actualiza la base de datos SQL Server mediante Entity Framework Core. Finalmente, el API devuelve una respuesta en formato JSON que Angular utiliza para actualizar la interfaz.



Arquitectura general del sistema Tienda Tecnológica

7. Base de datos

Nombre de la base de datos: TiendaTecnologicaDB

7.1. Tablas principales

Tabla	Descripción
Persona / Cliente / Empleado / Usuario	Datos de las personas que usan el sistema y sus cuentas de acceso
Producto / Categoria / Subcategoria / Marca	Catálogo de productos y su clasificación
Inventario / Bodega	Existencias de productos por bodega
Pedido / DetallePedido / EstadoPedido	Pedidos realizados por los clientes y sus productos
Pago / MetodoPago	Pagos asociados a cada pedido
Descuento / ProductoDescuento	Descuentos aplicados a productos
Garantia / ProductoGarantia	Garantías disponibles por producto
Proveedor	Proveedores de los productos
Devolucion	Devoluciones registradas sobre pedidos
ListaDeseos	Productos favoritos guardados por el cliente

7.2. Relaciones importantes

- Un Cliente puede tener varios Pedidos (1 a N).
- Un Pedido tiene uno o varios DetallePedido, cada uno asociado a un Producto (1 a N).
- Un Pedido tiene un EstadoPedido y puede tener un Pago asociado.
- Un Producto pertenece a una Subcategoria, que a su vez pertenece a una Categoria.
- Un Producto tiene existencias en Inventario, relacionadas con una Bodega.

7.3. Estructura general

La base de datos sigue un modelo relacional normalizado: las tablas de catálogo (Producto, Categoria, Subcategoria, Marca) están separadas de las tablas transaccionales (Pedido, DetallePedido, Pago), y ambas se relacionan mediante llaves foráneas. Esto permite mantener la integridad de los datos y facilita las consultas de reportes e historial.

8. Instalación y ejecución

8.1. Instalar los programas necesarios

- Instalar Visual Studio 2022 con la carga de trabajo 'Desarrollo de ASP.NET y web'.
- Instalar Node.js (versión LTS) y verificar con: `node -v`
- Instalar SQL Server y SQL Server Management Studio (SSMS).
- Instalar Git.

8.2. Crear / restaurar la base de datos

- Abrir SSMS y conectarse al servidor local.
- Ejecutar el script Database/script_completo.sql para crear la base de datos y sus tablas.
- Ejecutar (si aplica) los scripts de carga inicial: carga_catalogo_productos.sql, actualizacion_esquema_catalogo.sql y actualizacion_restricciones.sql.

8.3. Configurar la conexión

- Abrir el archivo Tienda.API/appsettings.json.
- Editar el valor de ConnectionStrings > DefaultConnection con el nombre de tu servidor y credenciales de SQL Server.

8.4. Abrir el proyecto

- Backend: abrir el archivo Tienda.slnx (o Ventas.sln) con Visual Studio.
- Frontend: abrir la carpeta Tienda/ con Visual Studio Code.

8.5. Ejecutar el backend

- En Visual Studio, seleccionar el proyecto Tienda.API como proyecto de inicio.
- Presionar F5 o el botón de ejecutar; el API quedará disponible (por defecto se abre Swagger para probarlo).

8.6. Ejecutar la aplicación (frontend)

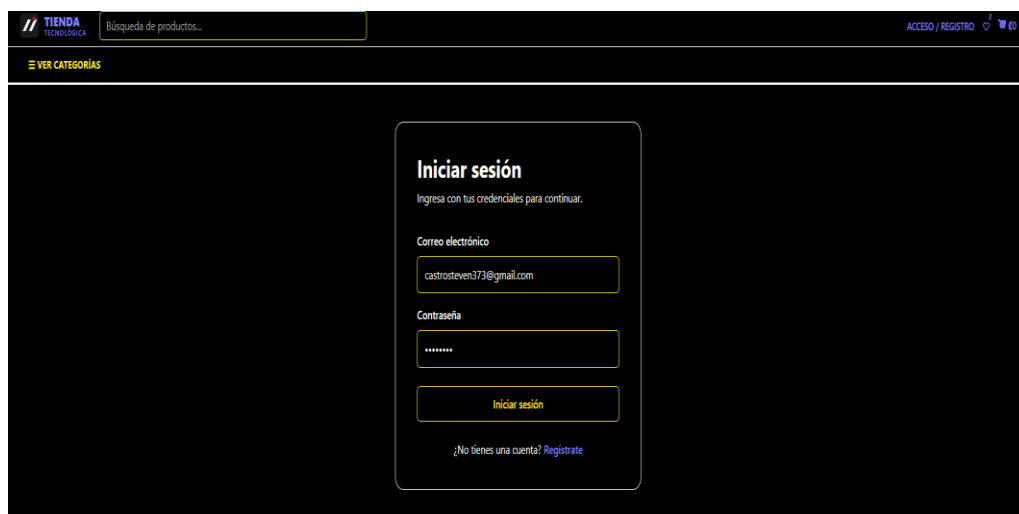
- Abrir una terminal dentro de la carpeta Tienda/.
- Ejecutar: npm install (solo la primera vez, para instalar dependencias).

- Ejecutar: `ng serve`
- Abrir el navegador en <http://localhost:4200>

9. Funcionalidades principales

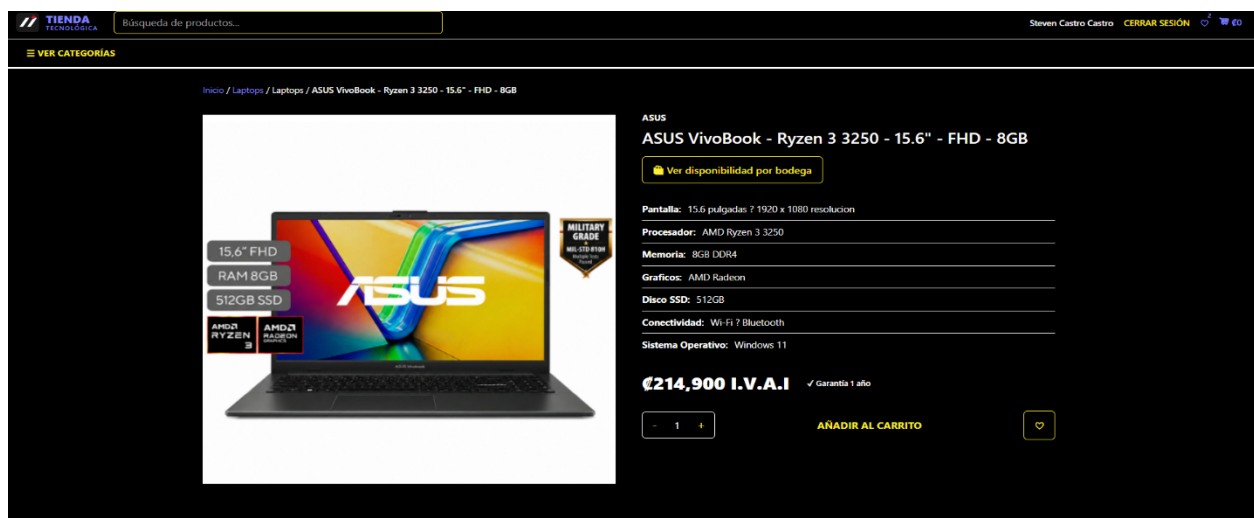
9.1. Inicio de sesión

Permite al cliente autenticarse con su correo y contraseña. El sistema valida las credenciales contra la base de datos y genera un token JWT que se utiliza para las siguientes peticiones.



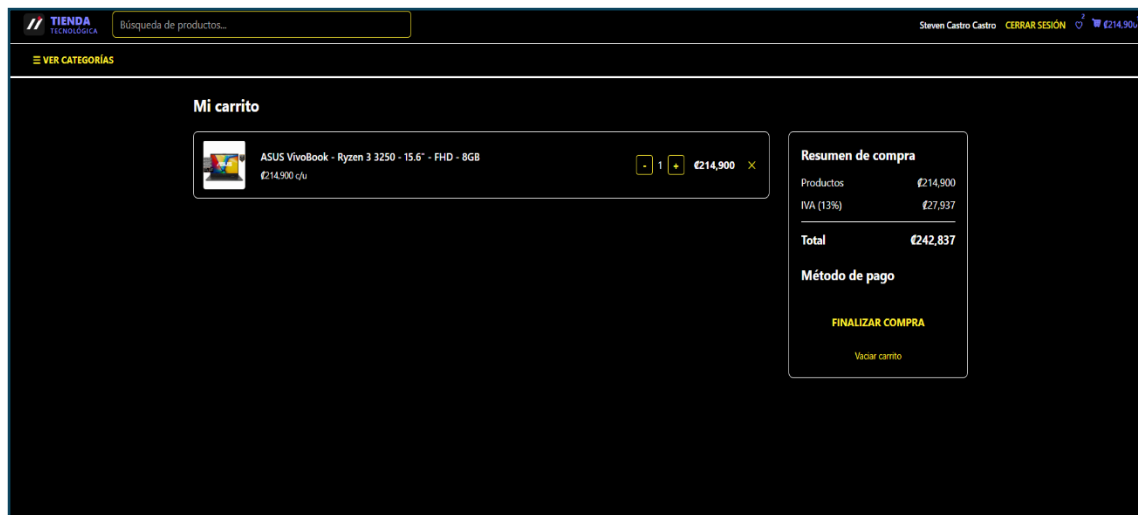
9.2. Catálogo y detalle de producto

Muestra la lista de productos disponibles, con opción de búsqueda y filtro por categoría. Al seleccionar un producto se muestra el detalle: imágenes, descripción, precio y existencias.



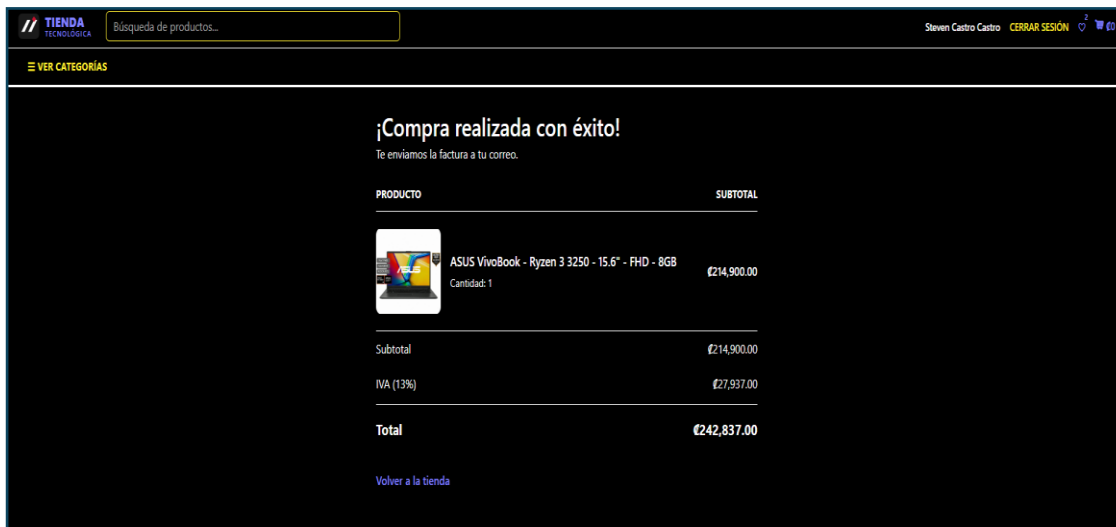
9.3. Carrito de compras

Permite agregar productos, modificar cantidades, quitar productos y ver el resumen de compra (subtotal, IVA y total) antes de finalizar el pedido.



9.4. Finalizar compra (checkout)

El cliente selecciona un método de pago y confirma el pedido. El sistema crea el Pedido, su Detalle y el Pago correspondiente en una sola operación, y muestra una confirmación con el número de pedido.



10. Pruebas

Prueba	Resultado esperado	Resultado
Inicio de sesión con datos correctos	Acceso al sistema y redirección a la tienda	Exitoso
Inicio de sesión con datos incorrectos	Mostrar mensaje de error de credenciales	Exitoso
Agregar producto al carrito	El producto se agrega y el total se actualiza	Exitoso
Finalizar compra sin método de pago	Mostrar aviso solicitando seleccionar un método de pago	Exitoso
Finalizar compra con datos válidos	Se crea el pedido y se muestra la confirmación	Exitoso

11. Mantenimiento y solución de problemas

11.1. El frontend no conecta con el API

Verificar que el backend (Tienda.API) esté en ejecución y que la URL configurada en los servicios de Angular apunte al puerto correcto del API.

11.2. Error de conexión a la base de datos

Revisar la cadena de conexión en appsettings.json (nombre del servidor, usuario y contraseña) y confirmar que el servicio de SQL Server esté iniciado.

11.3. Conflictos al fusionar ramas en Git

Antes de subir cambios, actualizar la rama local con git pull y resolver los conflictos localmente antes de hacer push, evitando así sobrescribir el trabajo de otros integrantes.

11.4. Error 401 (No autorizado) en el API

Verificar que el token JWT no haya expirado (la duración está configurada en appsettings.json) y que el usuario haya iniciado sesión correctamente.

12. Conclusiones

El desarrollo del sistema Tienda Tecnológica permitió cumplir con el objetivo planteado al inicio del proyecto: contar con una plataforma web funcional que centraliza el proceso de venta de productos tecnológicos, desde la exploración del catálogo hasta la confirmación del pedido y su pago. Se logró implementar una arquitectura en capas en el backend, un frontend dinámico en Angular y una base de datos relacional capaz de soportar la información de clientes, productos, inventario y pedidos de forma ordenada y consistente.

A lo largo del desarrollo, el equipo reforzó conocimientos en la construcción de APIs REST con .NET, el consumo de servicios HTTP desde Angular, el diseño de bases de datos relacionales y el uso de control de versiones con Git en un entorno de trabajo colaborativo. Este último punto en particular representó un aprendizaje importante, ya que trabajar en ramas separadas y luego integrar los cambios exigió coordinación entre los integrantes y buenas prácticas para evitar la pérdida de información.

Durante el proceso surgieron dificultades propias de un proyecto en equipo, como conflictos al fusionar código entre ramas de trabajo y ajustes necesarios en la comunicación entre

el frontend y el backend. Estas situaciones se resolvieron mediante la revisión conjunta del código, pruebas constantes del sistema y la definición de una estrategia clara para unificar los cambios en la rama principal, lo cual permitió avanzar sin perder el trabajo realizado por cada integrante.

Como mejoras futuras, el sistema podría ampliarse con funcionalidades adicionales como pasarelas de pago reales, un panel administrativo más completo con reportes y estadísticas de ventas, notificaciones automáticas al cliente sobre el estado de su pedido, y una optimización del rendimiento del catálogo para manejar un mayor volumen de productos.

13. Referencias

- Microsoft. Documentación de ASP.NET Core.
<https://learn.microsoft.com/aspnet/core>
- Microsoft. Documentación de Entity Framework Core.
<https://learn.microsoft.com/ef/core>
- Angular. Documentación oficial. <https://angular.dev>